

CRICGEEK

Expression Quality Score

Implementation & Technical Handover

A complete account of the EQS content-quality system, the Voice-to-Commentary workstream, and the database design — what the project is, what was built, and how to test every part of it.

Document	EQS Implementation & Technical Handover
Product	CricGeek Network LLP
Covers	EQS pipeline, Voice-to-Commentary, database design, testing guide
Status	Implemented — verified by typecheck, lint, and an offline self-test
Codebase	cricgeek-app (Next.js 16 / TypeScript / Prisma / SQL Server)
Pipeline version	eqs-pipeline-2.0.0
Config version	eqs-config-2026.09.1
Schema version	eqs-schema-1.0.0
Generated	28 September 2026

Read this first if you are new to the project

Part 1 explains what CricGeek is and why EQS exists. Part 6 is the testing guide and is written so it can be followed without reading anything else. For installing and running the app, see `SETUP.md` in the repository root — it is a copy-paste cheat sheet that assumes no prior knowledge of the codebase.

Contents

PART 1 The Project

1. What CricGeek is
2. Why EQS exists
3. System architecture
4. How an expression flows through the system

PART 2 The EQS System

5. What EQS is, and what it is not
6. Core principles, and where each is enforced
7. The eleven-stage pipeline
8. The thirteen evaluation dimensions
9. Writer DNA and mixed profiles
10. Final scoring methodology
11. Deterministic guardrails
12. Confidence and human review
13. The three specialist systems
14. Structured intermediate results
15. Model configuration

PART 3 Voice-to-Commentary

16. Pipeline and latency budget
17. Streaming speech-to-text

PART 4 Database Design

18. Tables, indexes, and retention

PART 5 What Was Built

19. Build-breaking bugs that were fixed
20. Bugs found by the self-test
21. Backend defects found and fixed
22. Complete file inventory

PART 6 Testing Guide

23. Test levels at a glance
24. Level 0 - Verify the install
25. Level 1 - Offline self-test (no keys needed)
26. Level 2 - Typecheck, lint, build

- 27. Level 3 - The health endpoint
- 28. Level 4 - API smoke tests
- 29. Level 5 - Behavioural tests of the principles
- 30. Level 6 - Benchmark and calibration
- 31. Level 7 - Database verification
- 32. Level 8 - Voice-to-Commentary
- 33. Regression checklist

PART 7 Caveats and Next Steps

- 34. Three things you must know
- 35. What is left to do

APPENDIX Reference

- A. Environment variables
- B. Command reference
- C. Troubleshooting

The Project

What CricGeek is, the problem EQS solves, and how the pieces fit together.

1. What CricGeek is

CricGeek is a cricket content platform. Alongside the things you would expect of a cricket site — live scores, a fixtures calendar, match previews and post-match analysis — its centre of gravity is a **community of cricket writers**. Registered writers publish opinion pieces, match reports, tactical breakdowns, and personal cricket stories, which the platform calls *expressions*.

Readers follow writers, save pieces, and react with a cricket-ball reaction. Writers accumulate a profile: an archetype, a level, badges, achievements, and contest standings. The platform also runs a live commentary feature where a creator speaks into a microphone and their commentary is published in near real time.

That community model creates a hard problem, and it is the problem this document is about. If writer status, contest results, and feed prominence are driven by content quality, then **something has to decide what “quality” means** — consistently, explainably, and at a volume no editorial team could read by hand.

The two workstreams covered here

Workstream	What it does	Where it lives
EQS Expression Quality Score	Evaluates the quality, credibility, expression, reasoning, originality, and suitability of a piece of cricket writing. Produces a score, a confidence, a dimension breakdown, evidence, and flags.	src/lib/eqs/
Voice-to-Commentary	Converts a creator's spoken cricket commentary into publish-ready live commentary while preserving meaning and style, inside a latency budget.	src/lib/voice/
Database design	Persists scores, component evidence, verified claims, plagiarism findings, model versions, Writer DNA history, and a full audit trail.	prisma/ + src/lib/eqs/db-ddl.ts

2. Why EQS exists

The naive approaches to scoring writing all fail in ways that matter on a cricket platform, and each failure would actively harm the community it is meant to serve:

Naive approach	Why it fails on CricGeek
Sentiment analysis	A blistering, well-argued critique of a selection policy is <i>excellent</i> cricket writing. Sentiment scoring would bury it and reward bland praise. Fans would learn that criticism is punished.
Grammar checking	Rewards clean prose that says nothing. A grammatically perfect 300 words of filler would outscore a rough-edged piece with a genuine tactical insight.
An AI detector	Detectors produce false positives on any careful, structured writer. Rejecting on that signal would expel exactly the writers the platform wants.
Asking one language model “is this good?”	The model will confidently hallucinate cricket statistics, cannot know whether text was copied from the corpus, and gives unstable answers run to run. Nothing about the verdict is auditable or reproducible.

EQS is the answer to all four. It is a **pipeline of specialist systems** whose structured results are combined by a deterministic scoring engine. Cricket statistics are verified against real cricket data, not guessed by a model. Plagiarism is checked by dedicated systems. Language judgement is the model's job and nothing else is. And every score carries the evidence and version information needed to explain it months later.

3. System architecture

Layer	Technology	Notes
Web application	Next.js 16 (App Router), React 19, TypeScript, Tailwind 4	Frontend and API routes in one deployment
Backend API	Next.js route handlers under <code>src/app/api/</code>	Node runtime; all EQS work is server-side
Database	Microsoft SQL Server via Prisma 5	Schema in <code>prisma/schema.prisma</code>
Authentication	NextAuth v5 (credentials + Google)	Roles: user, writer, admin
Primary AI model	Google Gemini (configurable)	Structured JSON output for language analysis
Fallback AI model	Self-hosted Ollama (Qwen)	Also drives BQS, tags, commentary polish
Cricket data	SportMonks API	Fixtures, live scores, scorecards, squads
Fact-check search	Tavily or Serper	For claims the scorecard cannot answer
Speech-to-text	Deepgram (Whisper service as fallback)	Batch today, streaming supported server-side
Python services	FastAPI (insights, RAG)	Optional; advanced match-insights pages only

Everything degrades, nothing hard-fails

Every external dependency above is optional except the database and auth. When a provider is missing or down, the affected stage returns a component result marked `unavailable`, the pipeline continues, and the final score comes back with **lower confidence** — which is the honest signal that less of the pipeline ran. `/api/health` reports exactly what is switched on.

4. How an expression flows through the system

1. A writer drafts a piece in the editor and presses publish.
2. The editor calls `POST /api/ai/eqs` to score the draft and shows the breakdown while publishing. This run is not persisted — drafts are scored repeatedly and only saved posts are worth an audit row.
3. `POST /api/blogs` creates the post, stores the linked match id (so statistics can later be checked against that match's scorecard), registers any contest entry, and writes a vector embedding of the text into the internal plagiarism corpus.
4. `POST /api/scoring/analyze` runs the legacy BQS pipeline and then the EQS pipeline, persisting both. EQS runs second deliberately: both need the fact-check search, so running BQS first warms a content-hash cache and the search is paid for once.
5. EQS results are written to `EqsRun` and its child tables, a summary is mirrored onto `BlogScore` so feed queries need no join, and the Writer DNA mix that drove the weighting is snapshotted for auditability.
6. If the run tripped a guardrail or came back with low confidence, it appears in `/api/eqs/review-queue` for an editor. It is **not** auto-rejected — publishing is never blocked by EQS.
7. Anyone can later read the stored breakdown from `GET /api/eqs/[blogId]`, with the original evidence, claim verdicts, and the model versions that produced it.

The EQS System

The pipeline, the dimensions, the weighting, and the guardrails.

5. What EQS is, and what it is not

EQS evaluates the quality, credibility, expression, reasoning, originality, and suitability of cricket content. The final score is produced **only after** the specialist component results are available.

EQS is not	Because
an AI detector	AI-assistance probability is one signal among thirteen dimensions, carries zero weight in the sum, and is capped at an 8-point deduction.
a sentiment score	The sentiment dimension scores whether the emotional direction <i>fits the argument in context</i> . Raw polarity is recorded as evidence only and never moves the score by itself.
a grammar checker	Expression quality is one dimension of thirteen, and is weighted differently for an analyst than for a storyteller.

6. Core principles, and where each is enforced

These are not aspirations in a document — each one is enforced by a specific structural decision in the code, which is what makes it hold under pressure.

Principle	Structural enforcement
No single signal decides whether an article is good or bad	Every guardrail is a cap , never a fixed score. An article that trips one still keeps the ordering its other dimensions earned below that cap.
Negative sentiment is not automatically poor quality	The sentiment dimension is defined as appropriateness in context, not polarity. The model prompt states this as a hard rule, and the benchmark measures it explicitly.
Positive wording is not automatically high quality	sarcasm_intent and evidence_quality are scored independently of tone, so vague praise cannot score well.
AI-generated probability is a signal, not a rejection rule	ai_assistance has weight 0 in every DNA weight table. It is structurally incapable of being a rejection rule, and its guardrail is a bounded deduction of at most 8 points.
Cricket statistics must be verified separately	The statistical_claims dimension is excluded from the language model's remit entirely and owned by the verification module. The prompt explicitly forbids the model from judging stat correctness.
Internal and external plagiarism are separate systems	Two modules, two component results, two database rows, two independent guardrails.
The final model must not invent stat or plagiarism findings	The interpretation stage receives only <i>normalised component results</i> , and its adjustment is clamped to a configurable maximum (default 6 points).

7. The eleven-stage pipeline

#	Stage	Implementation
1	Article input and metadata	API route

#	Stage	Implementation
2	Writer DNA	dna.ts + dna-weights.ts
3	Fast checks (deterministic)	stages/fast-checks.ts
4	Language analysis	stages/language-analysis.ts
5	Claim extraction	stages/claim-extraction.ts
6	Cricket verification	stages/cricket-verification.ts
7	Internal plagiarism	stages/internal-plagiarism.ts
8	External plagiarism	stages/external-plagiarism.ts
9	Component normalisation	scoring-engine.ts
10	Final EQS (weight, interpret, guardrail)	scoring-engine.ts + stages/final-interpretation.ts
11	Output and persistence	service.ts + persistence.ts

Concurrency and ordering

Ordering is load-bearing. Stages 4, 5, 7, and 8 run **concurrently**. Stage 6 waits on stage 5, because it verifies the claims stage 5 extracted. Stage 10 waits on everything, because it can only weigh evidence that exists.

```

      +-- 4 Language analysis  --+
3 Fast checks  --> | 5 Claim extraction  --> 6 Cricket verification --+
2 Writer DNA    | 7 Internal plagiarism --+                               |
      +-- 8 External plagiarism --+                               |
                                                              v
          9 Normalise --> 10a DNA weighting --> 10b Model (+/- 6) --> 10c Guardrails
                                                              |
                                                              v
                          11 Final EQS + confidence + explanation + flags

```

Failure behaviour

Each stage is wrapped in its own timeout, and the whole pipeline has an outer ceiling on top of that (the per-stage budgets can sum to more than a serverless invocation allows). A stage that fails or times out returns a degraded component result with a flag, and the pipeline continues.

If no language provider answers at all, the deterministic fast-check heuristics supply every dimension. **That path is the floor, not the target.** It cannot read sarcasm in context, and it cannot detect fabricated statistics or copied text, because those come from the specialist systems. Confidence drops accordingly, which is the point.

8. The thirteen evaluation dimensions

All dimensions are **0–100 and higher is always better**. That uniformity matters: it means a toxicity score of 100 signals no personal attacks, and no part of the engine needs to remember which scales are inverted.

Key	Dimension	Owned by
originality	Own information, interpretation, framing, expression	language analysis, scaled by both plagiarism systems
expression_quality	Clarity, readability, precision, sentence quality	language analysis
coherence	Logical flow, paragraph relationships	language analysis

Key	Dimension	Owned by
relevance_focus	Topic connection; avoids filler and repetition	language analysis
sentiment	Whether emotional direction fits the argument in context	language analysis
sarcasm_intent	Sarcasm, mockery, literal vs intended meaning	language analysis
toxicity	Insults, harassment, targeted attacks (100 = none)	language analysis + fast checks
constructive_criticism	Explains a weakness vs attacking a person	language analysis
reasoning	Argument quality, cause and effect, inference	language analysis
evidence_quality	Whether claims are <i>supported</i> (not whether true)	language analysis
statistical_claims	Whether claims are <i>correct</i>	cricket verification only
writer_dna_suitability	Success against the writer's intended type	language analysis
ai_assistance	AI-generation indicators (100 = reads as human)	fast checks + language analysis

Two deliberate exclusions: `statistical_claims` is outside the model's remit, and `ai_assistance` is outside the weighted sum.

9. Writer DNA and mixed profiles

EQS is DNA-aware because different cricket writers are trying to do different things. A storyteller's piece with no statistics in it is not a failed analyst piece, and should not be scored as one.

DNA	Typical purpose	Higher-priority dimensions
Fan	Passion, perspective, emotional connection	Originality, expression, coherence, relevance, constructive tone
Analyst	Data, tactics, technical interpretation	Stat accuracy, evidence, reasoning, relevance, clarity
Debater	Arguments, comparisons, viewpoints	Reasoning, evidence, constructive disagreement, coherence
Storyteller	Narrative, context, human angle	Originality, narrative flow, coherence, expression, engagement

Resolution order

1. An explicit `dnaoverride` (used by the benchmark runner to score the same text against different profiles).
2. The writer's stored `writerDNA` row. A brand-new writer sitting at the 25/25/25/25 default carries no information, so that case falls through.
3. Inference from the article text itself, for anonymous or first-time writers.

Mixed profiles are genuinely supported, not rounded away

The result is always a **normalised mix across all four profiles**, never a single archetype. `buildDnaWeights()` blends the four weight tables by that mix, so a writer who is 60% analyst and 40% storyteller gets genuinely blended weights. A profile needs 45% of the mix to be called dominant; below that the run is marked `mixed: true`. Blended weights are re-normalised to sum to exactly 1.

Weights live in `src/data/eqs-scoring-config.json`, **not in code**, and that file is versioned. Every stored run records which config version produced it, so a score from three months ago stays explainable even after the weights move.

10. Final scoring methodology

Article

- > Writer DNA
- > specialist analysis (stages 3-8, concurrent)
- > individual component results
- > normalisation (stage 9)
- > DNA/context weighting -> baseScore
- > final model interpretation -> bounded +/- EQS_MAX_MODEL_ADJUSTMENT (default 6)
- > deterministic guardrails -> capped
- > Final EQS + confidence + explanation + flags

Why the model runs before the guardrails

Two reasons, both non-negotiable. First, the model can never talk a score past a hard constraint — the guardrails get the last word. Second, its influence is clamped to a few points, so two runs of the same article cannot diverge meaningfully. That is the reproducibility guarantee, and it is why the adjustment is a bounded nudge rather than a free verdict.

11. Deterministic guardrails

Each guardrail is a *ceiling*. An article that trips one still ranks against other capped articles on the merits of its remaining dimensions.

Guardrail code	Trigger	Cap
TOXICITY_CAP	toxicity < 35, or explicit abuse detected	45
SPAM_CAP	spam / promotion patterns dominate	30
INTERNAL_EXACT_MATCH_CAP	at least 35% of the draft appears verbatim in CricGeek content	40
INTERNAL_DUPLICATE_CAP	vector similarity at or above 0.85	45
EXTERNAL_PLAGIARISM_CAP	provider matched 25% or more to outside sources	45
STAT_CONTRADICTION_CAP	at least 1 claim contradicted by trusted data	60
STAT_CONTRADICTION_CAP_SEVERE	3 or more claims contradicted	45
THIN_CONTENT_CAP	under 60 words	55
AI_ASSISTANCE_SIGNAL_PENALTY	ai_assistance < 45	bounded deduction, max 8 points

Guardrails are recorded even when they change nothing

If a score is *already* below a guardrail's cap, the guardrail is still recorded with a note saying no reduction was needed. Staying silent there would let an abusive or plagiarised article report “no deterministic guardrail needed” in its audit trail. This was a real bug caught by the self-test — see section 20.

12. Confidence and human review

Confidence is the weight-weighted mean of per-dimension confidence, reduced for each unavailable or errored component, and reduced again when a critical flag conflicts with an otherwise high score. It is the honest measure of how much of the pipeline actually ran.

A run is routed to `/api/eqs/review-queue` when:

- confidence falls below 0.5, or
- a guardrail demands it (toxicity, plagiarism, contradicted statistics), or
- sarcasm and intent are genuinely ambiguous, or
- the interpretation model itself asks for a human.

Flagging is not rejection

A toxicity, plagiarism, or AI-assistance signal **flags** a post for a human. It does not auto-reject it, and EQS never blocks publishing. This follows directly from the principle that no single signal decides whether an article is good or bad.

13. The three specialist systems

Cricket statistics verification

Claims are routed to trusted sources in order of authority:

1. The **match scorecard** from the cricket data API. Strongest evidence, but it requires the post to be linked to a match.
2. The **historical warehouse** and **web-search fact-check** pipeline for anything the scorecard cannot answer.

Verdicts are three-way: supported, contradicted, or inconclusive. **Only a contradiction pulls the score down.** “The scorecard has nothing on this player” is not evidence against the writer, and conflating the two would punish writers for gaps in our data. When nothing can be resolved at all, the dimension stays neutral at 75 and confidence drops — a writer should not lose points for our infrastructure being down.

Scorecard matching is shared with the legacy BQS pipeline via `lib/cricket-stat-match.ts`, so both scorers agree on what “verified” means.

Internal plagiarism

Both mechanisms the specification requires, used in sequence:

- **Vector similarity** — hashed embeddings over the `ExpressionEmbedding` table find the nearest neighbours cheaply.
- **Exact matching** — an 8-word shingle overlap against those neighbours' real text, which produces the actual matched passage as stored evidence. Eight words is long enough that natural overlap is rare.

The overlap ratio is asymmetric on purpose: it measures how much of *this* article is reused, not how similar two documents are overall.

External plagiarism

A genuinely separate system calling a dedicated third-party provider, storing the provider's own evidence rather than re-deriving a verdict locally. Unconfigured, it reports `unavailable` and the pipeline continues. Point `EXTERNAL_PLAGIARISM_API_URL` at your provider's scan endpoint, or at a thin adapter of your own; the response reader accepts the field names used by the common vendors.

14. Structured intermediate results

Every module returns the same shape, so the scoring engine only ever consumes structured data — never prose.

Field	Purpose
component	Module name

Field	Purpose
score	Normalised 0-100, or null where the module produces no score
confidence	0-1 confidence in this module's own result
evidence	Module-specific supporting evidence
flags	Issues requiring attention, each with a severity
claims	Extracted claims and their verification state
source	Provider or module that produced the result
version	Model, rule, or API version
status	ok / unavailable / error / skipped
durationMs	Stage latency

15. Model configuration

The primary language-analysis model is Gemini, kept configurable so newer or cheaper models can be benchmarked later without touching business logic.

```
EQS_PROVIDER=auto           # auto | gemini | ollama | heuristic
GEMINI_API_KEY=...
GEMINI_EQS_MODEL=gemini-3.5-flash
GEMINI_EQS_FAST_MODEL=      # cheaper model for high-volume sub-tasks
```

auto walks Gemini, then Ollama, then heuristics, and uses the first that is configured. **Pin EQS_PROVIDER to a single value before comparing benchmark reports**, otherwise two reports may not have used the same model.

Verify the Gemini model id before deploying

GEMINI_EQS_MODEL defaults to gemini-3.5-flash because that is the id named in the technical specification. Google's published Flash line has used 1.5, 2.0, and 2.5 ids. **If the id does not exist in your project, every Gemini call fails and the pipeline silently falls back** to Ollama or to heuristics. After configuring, check /api/health and confirm checks.eqsPrimaryProvider reads gemini.

Voice-to-Commentary

Spoken commentary to published text, inside a measured latency budget.

16. Pipeline and latency budget

#	Stage	Function	Implementation
1	Microphone	Capture creator speech	Browser MediaRecorder
2	Streaming STT	Speech to text, low latency	voice/streaming.ts, deepgram.ts
3	Cleanup	Fix transcription damage without changing meaning	commentary-player-correction.ts
4	Transformation	Low-latency model turns transcript into commentary	commentary-polish.ts
5	Output	Send to the commentary API	/api/commentary/[sessionId]/entries
6	Target	About 15-30s end to end or faster	measured per stage, stored per run

Every run records `sttMs`, `cleanupMs`, `transformMs`, and `totalMs` separately, so a latency regression can be **attributed to a stage rather than guessed at**. The response includes:

```
{
  "stages": { "sttMs": 1840, "cleanupMs": 2, "transformMs": 1120, "totalMs": 3390 },
  "latencyMs": 3390,
  "targetMs": 30000,
  "withinTarget": true,
  "latencyWarning": false
}
```

Observed latency across stored runs is available from `getVoiceLatencyStats()`, which reports the mean for each stage plus p95 and max on the total, and the share of runs inside target. That makes the specification's target *measurable* rather than merely asserted.

Player-name accuracy

Wrong player names are the most damaging failure mode in live commentary, so correction happens in three places:

1. The session's squad list is passed to the speech-to-text provider as keyterms, so names are biased toward correct spellings *at transcription time*.
2. Deterministic roster-based correction runs on the raw transcript.
3. The same deterministic correction runs **again after** the model polish — because the model is the last thing to touch the text, and a name it reintroduced incorrectly would otherwise ship.

17. Streaming speech-to-text

Batch transcription pays an upload round-trip for every clip, and that dominates the latency budget. `post /api/commentary/stt-token` mints a short-lived, scoped provider credential so the browser can stream microphone

audio directly to the provider — the long-lived API key never leaves the server.

Status: server side complete, client not yet migrated

The commentary UI still uses the batch route, which continues to work unchanged, so this addition is purely additive. Adopting streaming is a frontend change: open the socket with the minted token, send audio frames, and post the final transcript. Disable the endpoint entirely with `VOICE_STREAMING_ENABLED=false`.

Database Design

What is stored, so that a score can be explained months later.

18. Tables, indexes, and retention

Area	Table	Holds
EQS	EqsRun	Final score, base score, confidence, band, flags, guardrails, explanation, DNA snapshot, timings, pipeline and config version
EQS	EqsComponentResult	Per-module score, confidence, evidence, flags, source, version, latency
Verification	EqsClaim	Claim text and type, player, metric, value, status, verification source, provider, evidence, timestamp
Plagiarism	EqsPlagiarismResult	Internal and external findings as separate rows: similarity, matched percent, matched post, matched passage, provider evidence
Models	EqsModelVersion	Provider, model, prompt version, latency, status per analysis stage
Audit	EqsAuditEvent	Processing events, failures, timeouts, retries, with stage and duration
Writer DNA	WriterDNAHistory	DNA classification snapshots, so a past score stays explainable after the live profile moves on
Voice	VoiceCommentaryJob	Transcript, cleaned transcript, commentary, providers and models, status, per-stage latency, target compliance
Plagiarism	ExpressionEmbedding	Vector index of published expressions, for internal duplicate detection

BlogScore also carries a denormalised EQS summary (eqs, eqsConfidence, eqsBand, eqsRunId, eqsRequiresReview, eqsVersion, eqsFlagsJson) so feed and profile queries do not need to join EqsRun.

One source of truth for the DDL

The DDL lives in `src/lib/eqs/db-ddl.ts`. `prisma/eqs_migration.sql` is **generated** from it by `npm run eqs:migration`, so the checked-in migration and the runtime bootstrap cannot drift apart. The same tables are also declared in `prisma/schema.prisma` so the ORM can read them.

Tables self-bootstrap on first use via an idempotent `ensureEqsTables()`, which means a freshly cloned checkout starts scoring without a separate migration step. Set `EQS_AUTO_MIGRATE=false` once you have applied the SQL yourself.

Indexes

Chosen around the actual query patterns: latest run for a post (`blogId`, `createdAt` DESC), runs per writer, cache lookups by `contentHash`, the human-review queue (`requiresHumanReview`, `createdAt` DESC), component health (`component`, `status`), claim triage, and voice latency reporting.

Retention

Why the audit table has no foreign key

EqsRun children cascade on delete. EqsAuditEvent deliberately has **no** foreign key to EqsRun — an event must be recordable even when the run row was never written, and that is precisely the case where you need the audit trail. Audit events and voice jobs are therefore not cascaded and need their own retention policy if volume becomes a concern.

What Was Built

The defects found and fixed, and the complete file inventory.

19. Build-breaking bugs that were fixed

The repository did not compile before this work

`npx tsc --noEmit` was failing on `src/lib/scoring.ts`, the existing BQS scorer. A bad find-and-replace had corrupted two pieces of it. `npm run build` could not have succeeded on a clean clone.

Location	Defect	Impact
<code>scoring.ts:404</code>	<code>detectSarcasticRidicule()</code> returned <code>lowEvidenceAttack</code> , a variable that was never declared.	A <code>ReferenceError</code> on every sarcasm check, and TS18004 at build time. Fixed by implementing it from how the two call sites use it.
<code>scoring.ts:281</code>	The <code>concerns?: string[]</code> field of the model response type had been overwritten with an unrelated string literal.	Type errors in three places, and the model's "concerns" output was silently unreadable.
<code>scoring.ts:5</code>	A header comment line clobbered by the same replace.	Cosmetic, fixed alongside.

Typecheck baseline moved from 88 errors to 81. The remaining 81 are all one unrelated class - see section 34.

20. Bugs found by the self-test

An offline self-test was written as part of this work (`npm run eqs:selftest`). It caught two real bugs in the new code, which is exactly why it exists:

Bug	Root cause	Fix
Blended mixed-profile weights summed to 1.0030 , not 1.	Rounding 13 weights independently defeated the re-normalisation step that was supposed to guarantee the invariant.	The residual is now placed on the heaviest weight, making "weights sum to 1" exact.
A guardrail whose score was <i>already</i> below its cap was not recorded at all.	The cap helper returned early when no reduction was needed, so an abusive post's audit trail read "no deterministic guardrail needed".	Every guardrail that fires is now recorded, with a note when no reduction was required.

21. Backend defects found and fixed

Area	Defect and impact
<code>/api/blogs</code> POST dropped <code>matchId</code> and <code>contestId</code>	The frontend sends both; the route ignored them. <code>matchTag</code> was never set, which means cricket statistics verification never had a scorecard to check against for any user-published post — it always fell back to heuristics. This was the single biggest thing blocking the verification workstream from working at all. Contest submissions were also never created, so those leaderboards were dead.
<code>withTimeout</code> did not enforce its own deadline	It aborted the signal but did not <i>race</i> the deadline, so a stage whose underlying library ignored the signal could hold the pipeline open indefinitely. Now <i>races</i> the deadline and reports a typed timeout.

Area	Defect and impact
EQS_PIPELINE_TIMEOUT_MS was configured but never applied	Now wired as an outer ceiling, because the per-stage budgets can sum to more than a serverless invocation allows.
Duplicate fact-check spend	BQS and EQS each called the paid fact-check search. Both now share a content-hash-keyed cache, so /api/scoring/analyze pays for the search once instead of twice.
.env.example did not exist	The documentation referenced it, and .gitignore's .env* rule would have excluded it anyway. Created fully commented, with an exception added so it actually ships.
Health endpoint was silent about EQS	Now reports eqsReady, eqsPrimaryProvider, eqsSchemaReady, eqsStatVerificationReady, voiceReady, and versions.
Code duplication	About 90 lines of JSON parsing and 100 lines of scorecard matching were duplicated; both extracted to shared modules so BQS and EQS cannot diverge.

22. Complete file inventory

New files

Group	Files
EQS core (src/lib/eqs/)	types.ts, config.ts, utils.ts, json.ts, cache.ts, cached-fact-check.ts, dna.ts, dna-weights.ts, scoring-engine.ts, pipeline.ts, service.ts, benchmark.ts, audit.ts, persistence.ts, db-ddl.ts, db-schema.ts
EQS providers	providers/types.ts, providers/gemini.ts, providers/ollama.ts, providers/index.ts
EQS stages	stages/fast-checks.ts, stages/language-analysis.ts, stages/claim-extraction.ts, stages/cricket-verification.ts, stages/internal-plagiarism.ts, stages/external-plagiarism.ts, stages/final-interpretation.ts
Voice (src/lib/voice/)	config.ts, pipeline.ts, persistence.ts, streaming.ts
Shared	src/lib/cricket-stat-match.ts
Data	src/data/eqs-scoring-config.json, src/data/eqs-benchmark.json
API routes	api/eqs/run, api/eqs/[blogId], api/eqs/benchmark, api/eqs/config, api/eqs/review-queue, api/commentary/stt-token
Database	prisma/eqs_migration.sql (generated)
Scripts	scripts/eqs-selftest.mts, scripts/generate-eqs-migration.mjs, scripts/setup.ps1, scripts/setup.sh
Docs and config	SETUP.md, .env.example, docs/generate_eqs_handover_pdf.py

Modified files

File	Change
src/lib/scoring.ts	Bug fixes; deduped onto shared modules; cached fact-check
src/lib/internal-originality.ts	Exposes candidate matches for exact-match pass
src/app/api/ai/eqs/route.ts	Rewritten onto the pipeline; legacy contract kept
src/app/api/commentary/transcribe/route.ts	Refactored onto the staged voice pipeline; latency reporting added

File	Change
src/app/api/blogs/route.ts	Persists matchId and contestId
src/app/api/scoring/analyze/route.ts	Runs and persists the EQS pipeline
src/app/api/health/route.ts	EQS and voice readiness checks
prisma/schema.prisma	Nine new models; EQS summary columns on BlogScore
package.json	setup, db:push, db:studio, eqs:selftest, eqs:migration, typecheck
.gitignore	Un-ignores .env.example
README.md	Rewritten around quick start and EQS
docs/05, 07, 10, 11, 13	Database, API reference, EQS, voice, configuration
src/lib/demo-data.ts	Type-only widening
src/app/writer/[id]/page.tsx	Type-only widening

The only two frontend touches are type annotations

demo-data.ts and writer/[id]/page.tsx received **type annotations only** — no UI or behaviour change. They were required: the page already read score.processingStatus, which the API returns but the local interface omitted, and that type error blocks npm run build. Flagged here because the brief was to leave the frontend alone.

Testing Guide

Eight levels, from a no-dependency smoke test to full calibration.

23. Test levels at a glance

Work down the table. Each level needs strictly more setup than the one above it, so stop wherever your environment runs out.

Level	What it tests	Needs	Time
0	The install works	Node 20+	1 min
1	Scoring maths and safety invariants	Nothing (no keys, no database, no network)	10 s
2	Types, lint, production build	Prisma client generated	2-4 min
3	What is actually configured	App running	1 min
4	Every endpoint responds correctly	App running + database	10 min
5	The principles hold on real text	App + database + a language model	20 min
6	Calibration accuracy and false-positive rates	App + database + model + admin login	5-30 min
7	Data is persisted and auditable	Database access	10 min
8	Voice pipeline and latency	Deepgram key	10 min

24. Level 0 - Verify the install

```
cd cricgeek-app
node --version      # expect v20 or higher
npm install
```

Expected: dependencies install with no errors. If `npm` is not recognised, install Node.js LTS and reopen the terminal.

25. Level 1 - Offline self-test (no keys needed)

Start here. This is the fastest meaningful check and needs nothing configured. It runs the deterministic scoring core against all 36 benchmark samples with no database, no API keys, and no network, and asserts the invariants that must hold regardless of which provider is in use.

```
npm run eqs:selftest
```

It enforces seven properties:

1. Every DNA weight table sums to exactly 1.
2. Blended mixed-profile weights sum to exactly 1.
3. **Reasoned criticism is never scored as toxic.** This is the specification's headline principle, checked on every sample labelled as constructive criticism.

4. Explicit abuse always trips the toxicity guardrail.
5. Spam patterns always trip the spam guardrail, and clean cricket writing never does.
6. Scores stay within 0-100, and guardrails only ever *lower* a score.
7. The AI-assistance signal never costs more than its configured cap.

Expected output (tail)

```
id          category          exp   eqs   band        tox   sarc   ai   guardrails
eqs-hq-001  high-quality-human  high  67    publishable  96    88    92   -
eqs-crit-001 strong-criticism    high  59.2  publishable  96    88    78   -
eqs-tox-001  toxicity           low   45    needs-work   6     88    92   TOXICITY_CAP
eqs-vague-001 vague-writing       low   30.9  below-bar    96    88    92   -
...
36 samples scored, 0 assertion failure(s).
Self-test PASSED (deterministic fallback path).
```

How to read that table honestly

The incorrect-stats and copied-text rows score *well* here, and that is correct. This run has no verification or plagiarism specialist attached, and language heuristics genuinely cannot tell a fabricated statistic from a real one. That is the whole architectural point: those signals come from dedicated systems, not from reading the prose. To test them you need Level 5.

26. Level 2 - Typecheck, lint, build

```
npx prisma generate      # required first - see the caveat below
npm run typecheck
npm run lint
npm run build
```

Expect typecheck noise if prisma generate cannot run

If your network blocks binaries.prisma.sh (common on corporate networks, and the case during this implementation), prisma generate fails and you will see roughly 81 errors. They are all one class — TS7006 implicit-any on Prisma callbacks and TS2694 missing Prisma namespace types — and all of them resolve once the client generates. **None are in the EQS or voice code.** Confirm that with the filter below.

```
# Should print nothing at all:
npm run typecheck 2>&1 | grep -E 'lib/eqs|lib/voice|cricket-stat'
```

27. Level 3 - The health endpoint

```
npm run dev
# then open, or curl:
curl http://localhost:3000/api/health
```

This is the fastest way to see what is actually wired up. The fields that matter:

Field	Should read	If it does not
checks.database	true	DATABASE_URL is wrong, or SQL Server is not running
checks.eqSchemaReady	true	Run npm run db:push

Field	Should read	If it does not
checks.eqsReady	true	Set GEMINI_API_KEY, or run Ollama locally
checks.eqsPrimaryProvider	gemini	heuristic means no AI model is configured and scores will be flat
checks.eqsStatVerificationReady	true	Set SPORTMONKS_API_TOKEN or TAVILY_API_KEY
checks.eqsExternalPlagiarismConfigured	true (optional)	Set EXTERNAL_PLAGIARISM_API_URL; stage 8 stays unavailable without it
checks.voiceReady	true	Set DEEPGRAM_API_KEY

ok: false at the top only means one of the three core checks is off. The endpoint still responds and tells you which.

28. Level 4 - API smoke tests

Score a draft (no sign-in required)

```
curl -X POST http://localhost:3000/api/ai/eqs \
  -H "Content-Type: application/json" \
  -d '{"title":"Hard lengths","content":"Jasprit Bumrah barely bowled a yorker tonight, and that is exactly why the chase collapsed. He bowled the hard length into the surface and kept dragging the batters across the crease. Between overs eleven and sixteen the false-shot rate climbed because nobody could get under the ball."}'
```

Verify in the response:

- eqs, band, and confidence are present and sane.
- dimensions has **13** entries and the weights sum to 1.
- components has **6** entries, each with a status.
- overallEqs, weightedEqs, and attributes are present — these are the **legacy fields the publish UI reads**, and they must never disappear.
- modelVersions names the provider and model that actually served the run.

The other endpoints

Endpoint	Method	Access	Check
/api/eqs/run	POST	author or admin	Returns a score and persistence.persisted: true
/api/eqs/[blogId]	GET	any	Returns the stored run; add ?audit=1 as the author for the event trail
/api/eqs/config	GET	admin	Effective provider, model, weights, thresholds, versions
/api/eqs/review-queue	GET	admin	Runs flagged for a human
/api/eqs/benchmark?coverage=1	GET	open	Dataset coverage; costs nothing
/api/scoring/analyze	POST	any	Returns both bqs and an eqs block

Negative cases worth confirming

Request	Expected
Empty or very short content	400
Content over 60,000 characters	413
/api/eqs/run without sign-in	401
/api/eqs/run for someone else's post as a non-admin	403
/api/eqs/[blogId] for an unscored post	404 with a message telling you to score it
/api/eqs/benchmark as a non-admin	403
Voice transcribe with no provider configured	503 TRANSCRIPTION_SERVICE_UNAVAILABLE

29. Level 5 - Behavioural tests of the principles

This is the level that matters most, and it needs a language model configured (check eqsPrimaryProvider first). Each test below targets one specification principle. Post each text to /api/ai/eqs and check the named field.

#	Test	Input	Pass condition
1	Reasoned criticism is not abuse <i>The single most important test</i>	A harsh but evidence-led critique. Use eqs-crit-001 from the benchmark file.	toxicity at or above 60, no TOXICITY_CAP guardrail, and the overall score stays high
2	Explicit abuse is caught	eqs-tox-001	toxicity below 35 and TOXICITY_CAP present
3	Sarcastic mock praise is caught	eqs-sarc-001	sarcasm_intent below 45; score does not read as high quality
4	Positive wording alone does not score well	eqs-vague-001	relevance_focus and evidence_quality both low; band is below-bar or needs-work
5	Storyteller with no statistics is not punished	eqs-hq-002 with dnaOverride: {storyteller: 80, fan: 20}	Score stays high; statistical_claims has little weight in the blend
6	Fabricated statistics are contradicted	eqs-stat-001. Requires SPORTMONKS_API_TOKEN or TAVILY_API_KEY.	At least one claim with status: contradicted and a STAT_CONTRADICTION_CAP guardrail
7	AI-style prose is flagged but not rejected	eqs-ai-001	ai_assistance below 45; the penalty in guardrails is at most 8 points
8	Clean human writing is not falsely flagged as AI	eqs-ai-003	ai_assistance at or above 45; no AI penalty
9	Internal plagiarism is caught	Publish eqs-hq-001, then score eqs-copy-001 (a verbatim copy)	INTERNAL_EXACT_MATCH flag, a matchedPassage in the evidence, and a cap
10	Thin content is capped	A 20-word post	THIN_CONTENT_CAP present, score at or below 55
11	Low confidence routes to review	Score anything with EQS_PROVIDER=heuristic	requiresHumanReview: true with a reason naming confidence
12	Graceful degradation	Set GEMINI_API_KEY to a bad value and stop Ollama	Still returns 200 with a score; confidence drops; a LANGUAGE_ANALYSIS_FALLBACK flag appears

Sample texts for tests 1-10 are all in src/data/eqs-benchmark.json under the ids given, so you can copy them directly rather than writing your own.

30. Level 6 - Benchmark and calibration

Admin sign-in required; a full run consumes real model and search quota.

```
curl "http://localhost:3000/api/eqs/benchmark?coverage=1" # free
curl "http://localhost:3000/api/eqs/benchmark?limit=10&fast=1" # quick smoke
curl "http://localhost:3000/api/eqs/benchmark?category=sarcasm" # one category
curl "http://localhost:3000/api/eqs/benchmark" # full run
```

What the report tells you

Field	Meaning	What to watch for
summary.bandAccuracy	Share of samples landing in the right quality band	Track it across changes; treat a drop as a regression
summary.meanAbsoluteError	Average distance from the band midpoint	Lower is better
summary.meanConfidence	Average confidence	Low values mean components are not running
criticismNotPenalised	Reasoned criticism not scored as abuse	failedIds must be empty. Anything here is a direct violation of the headline principle
detectionMetrics.*.falsePositiveRate	Wrongly flagged, per signal	Watch toxicity and aiAssistance especially — false positives here drive writers away
detectionMetrics.*.falseNegativeRate	Missed, per signal	Watch toxicity and plagiarism
observedModels	Which models actually served the run	Two reports are only comparable when this matches
warnings	Anything that makes the report less meaningful	Read these before trusting the numbers

Two dataset caveats when reading a report

The copied-text samples only trip plagiarism detection once their source articles are in the corpus with embeddings. The incorrect-stats samples need a cricket data or search key configured, otherwise there is no trusted source available to contradict them. Without those, the corresponding metrics will read as false negatives that are really just missing infrastructure.

31. Level 7 - Database verification

After running `/api/eqs/run` on a post, confirm the audit trail is real. Use `npm run db:studio` or SQL directly:

```

-- The run itself
SELECT TOP 5 id, blogId, eqs, confidence, band, requiresHumanReview,
    pipelineVersion, scoringConfigVersion, processingTimeMs
FROM EqsRun ORDER BY createdAt DESC;

-- Six component results per run, each with a status
SELECT component, score, confidence, status, source, version, durationMs
FROM EqsComponentResult WHERE runId = '<RUN_ID>';

-- Claims with their verdicts and evidence
SELECT claimText, claimType, status, verificationSource, provider, verifiedAt
FROM EqsClaim WHERE runId = '<RUN_ID>';

-- Internal and external plagiarism as separate rows
SELECT kind, flagged, maxSimilarity, matchedPercent, matchedTitle
FROM EqsPlagiarismResult WHERE runId = '<RUN_ID>';

-- The model that produced each stage
SELECT stage, provider, model, promptVersion, latencyMs, status
FROM EqsModelVersion WHERE runId = '<RUN_ID>';

-- The processing trail, including any failures
SELECT stage, eventName, status, message, durationMs
FROM EqsAuditEvent WHERE runId = '<RUN_ID>' ORDER BY createdAt;

-- The DNA mix that actually drove the weighting
SELECT userId, analyst, fan, storyteller, debater, source, reason
FROM WriterDNAHistory ORDER BY createdAt DESC;

-- Summary mirrored onto BlogScore (should match EqsRun)
SELECT blogId, bqs, eqs, eqsConfidence, eqsBand, eqsRequiresReview, eqsVersion
FROM BlogScore WHERE blogId = '<BLOG_ID>';

```

Also confirm the self-bootstrap works: drop the EQS tables, restart the app, score something, and check they are recreated. Then confirm `npm run eqs:migration` regenerates `prisma/eqs_migration.sql` with no diff.

32. Level 8 - Voice-to-Commentary

```

# Batch transcription
curl -X POST http://localhost:3000/api/commentary/transcribe \
  -F "audio=@clip.webm" -F "sessionId=<SESSION_ID>" \
  -H "Cookie: <YOUR_SESSION_COOKIE>"

# Streaming credential
curl -X POST http://localhost:3000/api/commentary/stt-token \
  -H "Content-Type: application/json" -d '{"sessionId":"<SESSION_ID>"}' \
  -H "Cookie: <YOUR_SESSION_COOKIE>"

```

Check	Pass condition
Stage latencies are reported	stages has all four fields and they roughly sum to totalMs
Latency budget is tracked	withinTarget is true for a short clip; latencyWarning appears past 15s
Player names are corrected	Say a garbled surname from the session squad; the roster name comes back
No invented names	Say a name not in the squad; expect a generic role term, not a hallucinated player
Persistence	A VoiceCommentaryJob row exists with both transcripts and the timings

Check	Pass condition
Clip size guard	A file over 12MB returns 413
Streaming token is short-lived	expiresInSeconds is around 60; the response is Cache-Control: no-store
No provider configured	Unset DEEPGRAM_API_KEY and AI_SERVICE_URL; expect a clean 503

33. Regression checklist

Run this before changing any production model, weight, or threshold — the specification requires benchmarking before such a change, and this is that gate.

1. Run the full benchmark and **save the report**.
2. Make the change in `src/data/eqs-scoring-config.json`.
3. Bump `configVersion` in that same file.
4. Run `npm run eqs:selftest` — it must pass with 0 failures.
5. Re-run the full benchmark and compare against the saved report.
6. Confirm `criticismNotPenalised.failedIds` is still empty.
7. Confirm no detection false-positive rate got materially worse.
8. Confirm `observedModels` matches the saved report, otherwise you are not comparing like with like.
9. Only then deploy.

Caveats and Next Steps

What you must know, and what is deliberately unfinished.

34. Three things you must know

1. The Gemini model id may not be real

GEMINI_EQS_MODEL defaults to `gemini-3.5-flash` because that is the id named in the technical specification. Google's published Flash line has used 1.5, 2.0, and 2.5 ids. If this id does not exist in your project, **every Gemini call fails and the pipeline silently falls back** to Ollama or heuristics, which will look like "EQS works but scores are flat" rather than like an error.

Action: after configuring, open `/api/health` and confirm `checks.eqsPrimaryProvider` reads `gemini`. Override the variable if your project exposes a different id.

2. The benchmark set has 36 samples; the specification targets 200-500

The harness, the metrics, and the labelling schema are complete and production-ready. Growing the set to target is **editorial work**: the specification asks for human/editorial ground truth, which is precisely the thing that cannot be generated by the system being measured without becoming circular.

To extend it, append objects to `src/data/eqs-benchmark.json` using the existing shape. `groundTruth.band` is the editorial quality judgement, and the boolean fields drive the false-positive and false-negative metrics. `describeBenchmarkCoverage()` reports progress against the 200 target, broken down by category and by dominant DNA.

This is the one part of the specification not delivered in full, and the omission is deliberate.

3. Typecheck noise from a blocked Prisma download

During this implementation the network blocked `binaries.prisma.sh`, so `prisma generate` and `next build` could not run. That is why roughly 81 typecheck errors remain: every one is the same class and all resolve once the client generates. **Zero are in the new code.**

What was verified, and how

Check	Result
Pre-existing build-breaking bugs in <code>scoring.ts</code>	Fixed; baseline moved from 88 to 81 errors
Typecheck errors in new EQS or voice code	Zero (verified by filter)
ESLint across all changed files	0 errors (2 pre-existing warnings, unrelated)
Offline self-test	36 samples, 0 assertion failures
Generated SQL migration	Regenerates from the DDL source with no diff
Circular imports	None introduced (verified)
Full production build	Not verified - blocked by the Prisma download
Live model calls, real database writes	Not verified - no keys or database available in that environment

On a machine with normal network access, run `npx prisma generate` and the typecheck should be clean. Then work through Levels 2 to 8 of the testing guide.

35. What is left to do

Item	Why it matters	Effort
Grow the benchmark set to 200-500 labelled samples	Turns calibration from indicative into trustworthy. The gate on tuning any weight with confidence.	Editorial
Confirm and pin the Gemini model id	Prevents a silent fallback that looks like working software.	Minutes
Configure an external plagiarism provider	Stage 8 is the only specialist system with no live backend.	Small
Migrate the commentary UI to streaming STT	The server side is done. This is where the real latency win is.	Frontend
Calibrate weights and thresholds against the grown benchmark	Current values are reasoned defaults, explicitly not calibrated, exactly as the specification instructed at planning stage.	Analysis
Build an editor UI over the review queue	The endpoint and the flagging logic exist; there is no screen yet.	Frontend
Add a retention policy for audit events and voice jobs	Neither cascades, so both grow without bound.	Small

A closing note on what “done” means here

The pipeline, the specialist systems, the database design, the audit trail, the guardrails, the benchmark harness, and the voice latency instrumentation are implemented and verified to the extent the environment allowed. What is not done is **calibration** — and that is correct. The specification was explicit that exact weights and thresholds should be calibrated against an editorial benchmark rather than hard-coded at planning stage. The numbers shipped are documented, versioned, and reasoned defaults, sitting behind a harness built to replace them with measured ones.

Reference

Environment variables, commands, and troubleshooting.

A. Environment variables

`.env.example` in the repository root is the authoritative, fully commented list. This is the summary.

Required

Variable	Purpose
DATABASE_URL	SQL Server connection string
AUTH_SECRET	Session signing secret
AUTH_URL	Base URL of the app (AUTH_TRUST_HOST=true covers local dev)

EQS

Variable	Default	Purpose
EQS_PROVIDER	auto	auto gemini ollama heuristic
GEMINI_API_KEY	-	Primary language-analysis model
GEMINI_EQS_MODEL	gemini-3.5-flash	Model id - verify this one
GEMINI_EQS_FAST_MODEL	same as primary	Cheaper model for sub-tasks
OLLAMA_URL	localhost:11434	Self-hosted fallback model
EXTERNAL_PLAGIARISM_API_URL	-	Stage 8; unset disables it
INTERNAL_DUPLICATE_THRESHOLD	0.85	Vector similarity flag threshold
EQS_MAX_MODEL_ADJUSTMENT	6	Cap on the model's influence, in points
EQS_RUN_ON_ANALYZE	true	Run EQS in the publish flow
EQS_AUTO_MIGRATE	true	Create EQS tables on first use
EQS_PERSISTENCE_ENABLED	true	Store runs and the audit trail
EQS_CACHE_TTL_MS	600000	Component result cache lifetime

Cricket data and voice

Variable	Purpose
SPORTMONKS_API_TOKEN	Fixtures, live scores, scorecards. Needed to verify stats
TAVILY_API_KEY / SERPER_API_KEY	Fact-check search for claims the scorecard cannot answer
ALLOW MOCK MATCH DATA	Serve mock fixtures; keep false in production
DEEPGRAM_API_KEY	Speech-to-text
VOICE_LATENCY_TARGET_MS	End-to-end budget, default 30000

Variable	Purpose
VOICE_STREAMING_ENABLED	Allow browser-direct streaming via a minted token

B. Command reference

Command	What it does
npm run dev	Start the app at localhost:3000
npm run build	Production build (also typechecks)
npm run typecheck	Types only
npm run lint	Code style
npm run db:push	Create or update all database tables
npm run db:studio	Visual database browser at localhost:5555
npm run eqs:selftest	Offline scoring-engine check - no keys, no database
npm run eqs:migration	Regenerate prisma/eqs_migration.sql from the DDL
npm run setup	install + prisma generate + db push
scripts/setup.ps1 (Windows)	One-shot guided setup
scripts/setup.sh (Mac / Linux)	One-shot guided setup

C. Troubleshooting

Symptom	Cause	Fix
npm is not recognised	Node.js not installed, or a stale terminal	Install Node LTS, reopen the terminal
Cannot find module '@prisma/client'	Client never generated	npx prisma generate
unable to get local issuer certificate	Corporate network blocking binaries.prisma.sh	Allow that host or set HTTPS_PROXY. The app still runs; EQS creates its own tables at runtime
Can't reach database server	SQL Server down or wrong URL	Start the database, re-check DATABASE_URL
Invalid object name 'dbo.Blog'	Tables never created	npm run db:push
EADDRINUSE port 3000	Port already in use	npm run dev -- -p 3001
EQS scores all cluster around 60	No AI model configured, so the deterministic fallback is scoring	Set GEMINI_API_KEY or run Ollama. Check /api/health eqsPrimaryProvider
Stat claims never verified	No cricket data source, or the post is not linked to a match	Set SPORTMONKS_API_TOKEN and link a match when publishing
Plagiarism never detected	Corpus has no embeddings yet, or no external provider	Publish a few posts first; set EXTERNAL_PLAGIARISM_API_URL
Voice returns 503	No transcription provider	Set DEEPGRAM_API_KEY
Blank page, chunk errors in console	Stale build cache	Delete the .next folder and restart

Further reading in the repository: [SETUP.md](#) (setup cheat sheet) • [docs/10-EQS.md](#) (EQS detail) • [docs/11-VOICE-TO-COMMENTARY.md](#) • [docs/05-DATABASE.md](#) • [docs/13-CONFIGURATION.md](#) • [.env.example](#)